

# TestFlow Revenue Architecture

## Subscription Plans & Revenue Models

Date

2026-05-30

Author

Parakh Sharma (solo founder)

Status

Draft for review

Related

PRICING.md, SALES\_TARGETS.md, financial-plan .tex

### 1. Why this document

PRICING.md commits to one sound idea: a **flat per-workspace annual fee**, no per-seat metering. Keep that as the *spine*. But a single flat fee leaves money — and customers — on the table at both ends of the market: the seasonal two-substation contractor who will never sign an annual contract, and the utility that wants a framework license for 40 sites.

This document layers **seven revenue models** onto that spine so every buyer has a way in and every customer has a way to spend more over time. It is grounded in what the database already supports: `subscriptions.plan_id` (free-text SKU), `subscriptions.seat_count`, `companies.features` (JSONB feature flags read by `has_feature()` / `useFeature()`), and the Razorpay webhook (`upsert_subscription`).

**Design principle — Land, Expand, Monetize the edges.** The annual subscription is the *land*. Metered add-ons and seat-free capacity tiers are the *expand*. Consumption packs, outcome pricing, white-label, and utility framework deals *monetize the edges* of the market the flat fee cannot reach.

### 2. The revenue stack at a glance

| # | Revenue model               | What the customer buys                      | Recurring?      |
|---|-----------------------------|---------------------------------------------|-----------------|
| 1 | Core subscription ladder    | Annual workspace access & tier features     | Yes (annual)    |
| 2 | Hybrid metered add-ons      | Usage beyond fair-use (AI, storage, comms)  | Yes (variable)  |
| 3 | Consumption / project packs | Prepaid “project credits”, no contract      | No (repeat)     |
| 4 | Outcome / value pricing     | Per signed handover / commissioned bay      | Yes (variable)  |
| 5 | Channel & white-label       | OEM/EPC resells to subcontractors           | Yes (wholesale) |
| 6 | Utility framework license   | Multi-site, multi-year, on-prem option      | Yes (multi-yr)  |
| 7 | Services & data products    | Onboarding, migration, benchmarks, training | Mixed           |

A customer typically starts in one model and accretes others. A mid-size contractor lands on [team\\_annual](#) (1), buys AI-report overages (2), takes a white-label add-on (7), and three years later signs a utility framework (6) when they win a PowerGrid package.

### 3. Model 1 — Core subscription ladder

The spine. Five rungs instead of the current four, adding a downmarket `starter_annual` so small contractors are not forced to choose between “₹ 2.4L” and “nothing”.

| Tier       | plan_id                        | Annual (INR)  | Active projects | Headline gating                     |
|------------|--------------------------------|---------------|-----------------|-------------------------------------|
| Free trial | <code>trial</code>             | ₹ 0 / 14 days | 1               | All templates, mobile, no AI        |
| Starter    | <code>starter_annual</code>    | ₹ 60,000      | 3               | Mobile + reports; no AI, no SSO     |
| Team       | <code>team_annual</code>       | ₹ 2,40,000    | unlimited       | + AI reports, audit-log UI, SLA 1 d |
| Business   | <code>business_annual</code>   | ₹ 6,00,000    | unlimited       | + custom templates, SSO, 4 h SLA    |
| Enterprise | <code>enterprise_custom</code> | ₹ 12,00,000+  | unlimited       | + CSM, on-prem, custom RBAC         |

**Unchanged rules from PRICING.md:** priced per workspace, not per seat; annual upfront is default; multi-tenancy, mobile, soft-delete and export are *never* gated. `starter_annual` deliberately excludes AI and SSO so it cannot cannibalise Team.

#### 3.1 Billing cadence as a price lever

Cadence is sold, not given. Each step toward upfront cash earns the customer a discount and earns you working capital.

| Cadence        | Adjustment       | Notes                                              |
|----------------|------------------|----------------------------------------------------|
| Annual upfront | list price       | Default. Best margin + float.                      |
| Quarterly      | +5%              | Paid concession for cash-flow-shy buyers.          |
| Monthly bridge | +0%, 90 days max | Trial→annual ramp only, never permanent.           |
| 2-year prepaid | −20%             | Locks logo + reduces churn risk.                   |
| 3-year prepaid | −30%             | Floor; never discount deeper without a concession. |

### 4. Model 2 — Hybrid metered add-ons

Flat platform fee *plus* a meter on the few resources whose cost scales with heavy use. This is the single biggest change to the revenue shape: it turns power users into more revenue without raising the headline price that wins deals.

| Metered SKU         | plan_id suffix          | Unit price     | Fair-use included            |
|---------------------|-------------------------|----------------|------------------------------|
| AI handover reports | <code>ai_report</code>  | ₹ 500 / report | 50/mo Team, unlt'd Business+ |
| Document retention  | <code>storage_gb</code> | ₹ 100 / GB-yr  | 25 GB Team, 100 GB Business  |
| SMS/WhatsApp alerts | <code>comms_pack</code> | ₹ 1 / message  | email always free            |
| Extra workspace     | <code>extra_ws</code>   | ₹ 90,000 / yr  | 1 per contract               |

**Mechanics.** Meters are counted against the `rate_limits` table that already backs `rate_limit_check`. AI reports are a natural meter — each generation is already concurrency-locked and rate-limited (10/hr). At month close, sum overages above fair-use and raise a Razorpay invoice line. Crucially, **fair-use is generous** so the meter is invisible to 90% of customers and only bites genuine power users — the ones who can afford it.

**Guardrail.** Never meter a *measurement*. Metering test records or equipment instances makes engineers under-report to dodge the meter — it corrupts the data you exist to protect. Meter only outputs (reports, storage, notifications) and capacity (workspaces), never field input.

## 5. Model 3 — Consumption / project packs

For the seasonal contractor who refuses an annual commitment: sell prepaid **project credits**. One credit unlocks one project workspace (full Team features) for the life of that commissioning job. Credits never expire.

| Pack   | plan_id              | Credits | Price (INR) | Per project |
|--------|----------------------|---------|-------------|-------------|
| Single | <code>pack_1</code>  | 1       | ₹ 35,000    | ₹ 35,000    |
| Crew   | <code>pack_5</code>  | 5       | ₹ 1,50,000  | ₹ 30,000    |
| Fleet  | <code>pack_20</code> | 20      | ₹ 5,00,000  | ₹ 25,000    |

**Why it works.** It converts the “I’ll wait until next month to start” objection into a purchase today, and it is a natural *on-ramp to annual*: a contractor who burns 8+ credits a year is shown the math — “you have spent ₹ 2.0L on packs; `team_annual` is ₹ 2.4L for unlimited.” The pack price per project is deliberately *higher* than the annual-implied rate so the upgrade always looks rational.

**Schema fit.** Credits live as a balance in `companies.features` (e.g. `project_credits: 5`); generating a project decrements it. No new table required for v1.

## 6. Model 4 — Outcome / value pricing (selective)

Tie a slice of price to the value event itself: a **signed, approved handover**. Offered only where a buyer resists fixed fees and wants “pay-for-results”. Two shapes:

- **Per commissioned bay** — ₹ 1,500 per equipment instance that reaches `APPROVED`. Predictable for the buyer, scales with their real workload. Cap it at the Team annual so it can never exceed the flat alternative — the cap is the upsell.
- **Per handover report delivered** — ₹ 2,000 per `CLOSED` project’s final report. Aligns spend with the deliverable the client actually pays *them* for.

Use sparingly. Outcome pricing maximises trust on the first deal but makes revenue lumpy and harder to forecast; migrate these accounts to flat annual once they trust the product (typically renewal 1).

## 7. Model 5 — Channel & white-label (B2B2B)

Let EPC majors and switchgear OEMs resell TestFlow to *their* subcontractors under their own brand. You sell wholesale; they own the relationship.

| Lever              | Terms                    | Rationale                                            |
|--------------------|--------------------------|------------------------------------------------------|
| Wholesale rate     | list $\times$ <b>0.6</b> | Partner keeps the margin + does the selling.         |
| White-label add-on | <b>₹</b> 1,00,000 / yr   | Custom domain, logo, colours ( <b>white_label</b> ). |
| Minimum commit     | 10 workspaces / yr       | Filters out tyre-kickers; guarantees volume.         |
| Co-marketing       | case study + logo rights | Lowers your CAC on the next direct deal.             |

**Schema fit.** Each reseller is a parent **company** record; sub-tenants carry **partner\_wholesale** as **plan\_id** and inherit the **white\_label** feature flag. Razorpay bills the *partner* a single consolidated invoice, not each sub-tenant. This is the highest-leverage channel because one signed OEM can deliver 20–50 workspaces with near-zero direct CAC.

## 8. Model 6 — Utility framework licensing

The top of the market: PowerGrid, NTPC, state transcos. They do not buy “a workspace” — they buy a **framework agreement** covering many sites for several years, often with on-prem or private-cloud requirements.

- **Site-block license** — published Enterprise rate  $\times$  0.7 for 20+ sites bought together (matches **SALES\_TARGETS** volume play).
- **Multi-year** — 3–5 year terms, annual escalation clause (CPI or flat 5%/yr) written in.
- **On-prem / private cloud** — premium of **₹** 5,00,000+/yr for a dedicated Supabase project or self-hosted deployment; gated by the **on\_prem** flag and a separate deployment SKU.
- **Contractual SLA & security review** — priced into the deal, not given away (each costs real founder/engineer time).

These are slow (6–12 month sales cycles) but anchor ARR and create reference logos that close everything below them.

## 9. Model 7 — Services & data products

Non-subscription revenue that improves margin and stickiness.

| Item                              | Price (INR)              | Type                                       |
|-----------------------------------|--------------------------|--------------------------------------------|
| Onboarding / setup fee            | <b>₹</b> 25,000 one-time | Team & Business; waivable as “discount”    |
| Data migration / import           | <b>₹</b> 25,000 one-time | Legacy Excel $\rightarrow$ TestFlow        |
| Custom report template            | <b>₹</b> 50,000 each     | Per template ( <b>custom_templates</b> )   |
| On-site training (8 h)            | <b>₹</b> 50,000 + travel | Per session                                |
| Premium support (4 h resp.)       | <b>₹</b> 2,00,000 / yr   | Upsell below Business SLA                  |
| <i>Benchmark / analytics pack</i> | <i>future</i>            | Anonymised, opt-in failure/throughput data |

The *data product* is the long-game option: with enough tenants, anonymised, opt-in commissioning benchmarks (mean time-to-handover, failure rates by equipment type) become a sellable industry

report — a revenue stream with *zero* marginal delivery cost. Strictly opt-in; multi-tenancy isolation (RLS) is never compromised for it.

## 10. Feature-gating matrix (maps to `companies.features`)

Every gate below is a boolean key in the `companies.features` JSONB, enforced by `has_feature()` server-side and `useFeature()` in the UI. Flags default TRUE unless a downgrade sets them false — so a missing flag never accidentally locks a paying customer out.

| Feature flag       | Starter | Team   | Business | Enterprise |
|--------------------|---------|--------|----------|------------|
| ai_reports         | —       | ✓      | ✓        | ✓          |
| audit_log_ui       | —       | ✓      | ✓        | ✓          |
| custom_templates   | —       | —      | ✓        | ✓          |
| sso                | —       | —      | ✓        | ✓          |
| white_label        | —       | add-on | add-on   | ✓          |
| priority_support   | —       | —      | ✓        | ✓          |
| advanced_analytics | —       | —      | —        | ✓          |
| on_prem            | —       | —      | —        | ✓          |

**Never gated (in every tier):** all test templates, mobile app, multi-tenant RLS isolation, real-time / 30s polling sync, soft-delete & restore, PDF + Excel export. Audit-log *capture* is always on (triggers); only the *viewer UI* is gated.

## 11. Expansion levers & the metric that matters

A flat annual fee alone gives ~100% net revenue retention at best (you only ever lose, never grow, within an account). The layered models above create **negative churn** — accounts that grow without re-negotiating the base:

1. **Tier upgrades** — Starter → Team when project cap bites.
2. **Metered overages** — AI/storage creep as usage deepens.
3. **Extra workspaces** — new business unit / region.
4. **Pack→annual conversion** — the consumption on-ramp.
5. **Add-ons** — white-label, templates, premium support.

**The single number to track: ARR per workspace** (from PRICING.md), now paired with **Net Revenue Retention**. If NRR > 100%, the expansion models are working and you can grow ARR *without* adding logos. Target NRR ≥ 110% by end of year 2.

## 12. Implementation notes (what the code already supports)

| Need                  | Already in schema / what to add                                                                                                                                   |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Plan identity         | <code>subscriptions.plan_id</code> (TEXT) — use the slugs above.                                                                                                  |
| Feature gating        | <code>companies.features</code> JSONB + <code>has_feature()</code> / <code>useFeature()</code> .                                                                  |
| Trial                 | <code>companies.trial_ends_at</code> (14-day default trigger).                                                                                                    |
| Metering              | <code>rate_limits</code> table + <code>rate_limit_check</code> RPC; sum overages monthly.                                                                         |
| Billing sync          | <code>razorpay-webhook</code> → <code>upsert_subscription()</code> (idempotent).                                                                                  |
| Seat count (optional) | <code>subscriptions.seat_count</code> exists but stays 0 — we do <i>not</i> price per seat.                                                                       |
| <b>To build</b>       | project-credit balance in <code>features</code> ; overage invoicing job; Razorpay plan objects per <code>plan_id</code> ; partner/parent-company billing roll-up. |

## 13. Anti-patterns (carry-over + new)

- Never meter test records or equipment instances — corrupts field data.
- Never gate multi-tenancy, mobile, or audit *capture* — core trust.
- Don't ship a permanent free tier; the trial + free template library is the lure.
- Don't let `starter_annual` include AI/SSO — it would cannibalise Team.
- Don't price packs *below* the annual-implied rate — the upgrade must always look rational.
- Don't tag Enterprise “most popular” — Team is the workhorse.
- Don't run outcome pricing forever — migrate to flat annual by renewal 1.

Companion docs: `PRICING.md` (strategy & display), `SALES_TARGETS.md` (named accounts), `financial-plan.tex` (unit economics & 36-month ARR). Review quarterly; update slugs here when Razorpay plan objects are created.